

8

ALGORITMOS MARL

ARQUITECTURAS MARL

- ✓ IQL (Independent Q-Learning, 1993)
- ✓ MADDPG (Multi-Agent Deep Deterministic Policy Gradient, 2017)
- ✓ VDN (Value Decomposition Networks, 2018)
- ✓ COMA (Counterfactual Multi-Agent Policy Gradients, 2018)
- ✓ QTRAN (Q-learning with TRANSformation, 2019)
- ✓ QMIX (2020)
- ✓ ROMA (Role-based Multi-Agent reinforcement learning, 2020)
- ✓ MAPPO (Multi-Agent PPO, 2022)

ARQUITECTURAS MARL

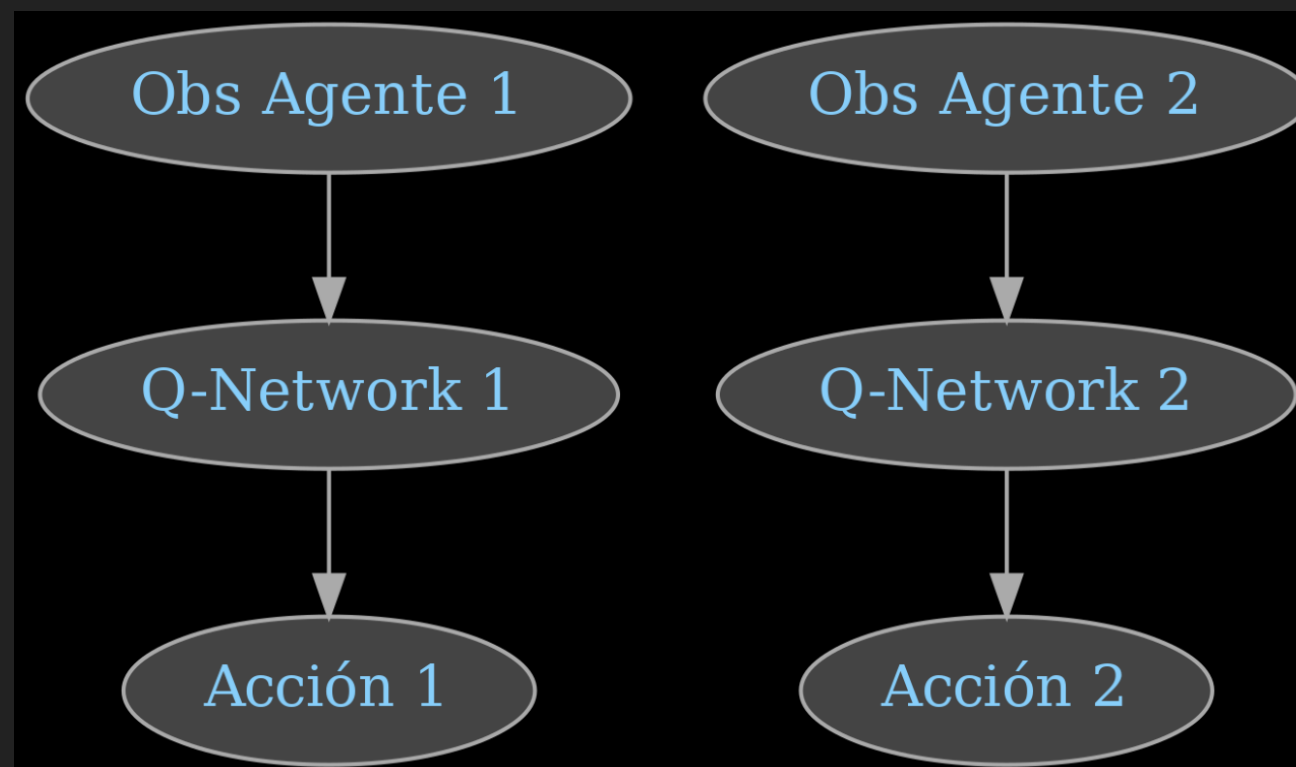
Algoritmo	Tipo
IQL	Mixto
MADDPG	Mixto
VDN	Cooperativo
COMA	Cooperativo
QTRAN	Cooperativo
QMIX	Cooperativo
ROMA	Cooperativo
MAPPO	Mixto

IQL



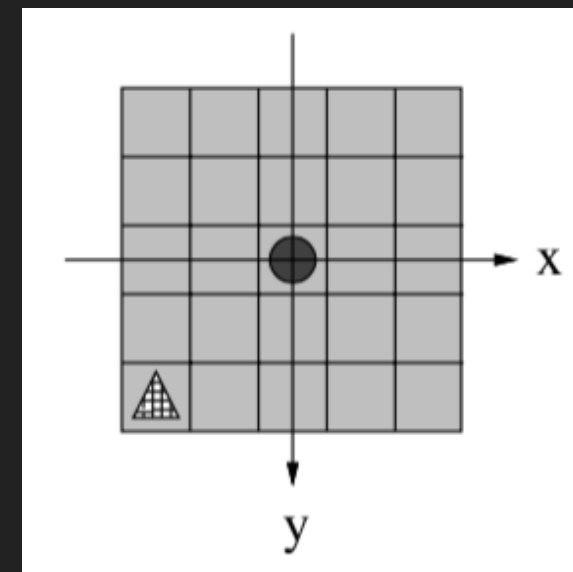
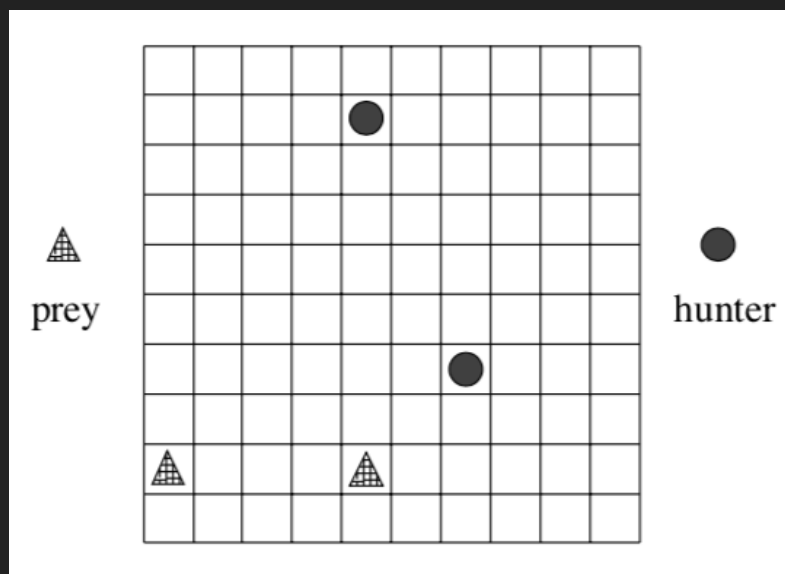
IQL (INDEPENDENT Q-LEARNING)

- ▶ Cada agente aprende por separado, como si fuera el único en el mundo.
- ▶ Cada agente no se comunica ni coordina con los demás.
- ▶ Dado un grupo de agentes ¿rinden mejor cuando cooperan entre ellos comparado con agentes que aprenden solos?.



ENTORNO IQL

- ▶ Se usó un gridworld de 12x12 con agentes cazadores que tienen que aprender a atrapar a otros agentes (presas) que se mueven.
- ▶ Movimiento agentes: arriba, abajo, izquierda, derecha.
- ▶ Visión del agente: 2, 3 y 5 casillas de distancia.



ENTORNO IQL

- ▶ El aprendizaje se basa en Q-Learning.
- ▶ Utiliza distribución de Boltzmann para tomar las acciones (Estado x , Acciones a_i , T is the temperature parameter that adjusts the randomness of decisions).
- ▶ Los agentes actualizan los valores Q con la recompensa inmediata que reciben: +1 si atrapan una presa, -0.1 por cada paso sin éxito.
- ▶ El éxito se mide con la cantidad de pasos que tardan en promedio en captura una presa.

$$p(a_i|x) = \frac{e^{Q(x,a_i)/T}}{\sum_{k \in actions} e^{Q(x,a_k)/T}}$$

IQL SIMPLE

- ▶ 1 cazador solo tarda 25.32 pasos, pero al azar 123.08 pasos.
- ▶ 2 cazadores 12.21 pasos aprendiendo contra 56.47 al azar.
- ▶ ¿La cooperación mejora esto aun más?



N-of-prey/N-of-hunters	1/1	1/2	1/2 (joint task)	2/2 (joint task)
Random hunters	123.08	56.47	354.45	224.92
Learning hunters	25.32	12.21	119.17	100.61

CASO 1 – COMPARTIR SENTIDOS

- ▶ Si los agentes se comunican lo que ven ¿ayuda o estorba?
- ▶ Se usó un **explorador dedicado** (le cuenta al cazador lo que ve).
- ▶ El cazador usa esa información extra si el no ve nada pero prioriza su propia vista.
- ▶ El espacio de estados del cazador se agranda, pero la visión extra superó el costo de la complejidad.

Hunter Visual Depth	Scout Visual Depth	Average Steps to Capture a Prey	
		Training	Test
2	no scouting	47.14 (± 1.28)	49.49 (± 1.60)
2	2	46.33 (± 1.39)	42.91 (± 1.48)
2	3	39.78 (± 1.06)	32.08 (± 1.22)
2	4	32.67 (± 1.03)	25.07 (± 0.89)

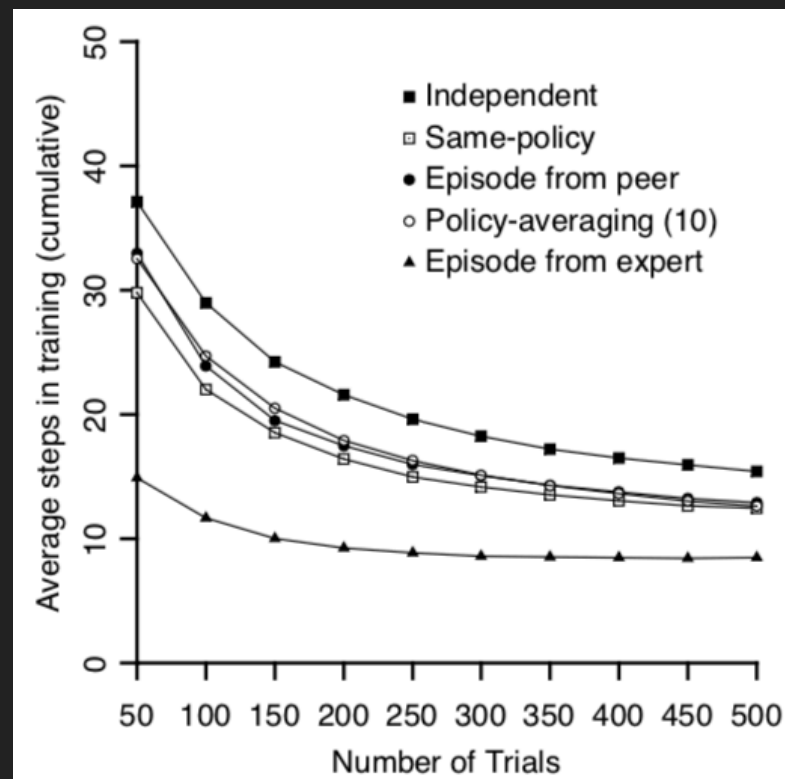
COOPERACIÓN ENTRE CAZADORES

- ▶ Consiste en 2 cazadores y una presa donde los cazadores comparten la visión.
- ▶ La exploración mutua supera a los independientes.

	Visual Depth	Average Steps to Capture a Prey	
		Training	Test
Independent agents	2	20.38 (± 0.57)	24.04 (± 1.00)
Mutual-scouting agents	2	25.20 (± 0.79) (worse)	24.52 (± 1.24) (same)
Independent agents	3	14.65 (± 0.53)	16.04 (± 0.56)
Mutual-scouting agents	3	14.02 (± 0.75) (same)	12.98 (± 0.65) (better)
Independent agents	4	12.21 (± 0.65)	11.53 (± 0.61)
Mutual-scouting agents	4	11.05 (± 0.56) (better)	8.83 (± 0.78) (better)

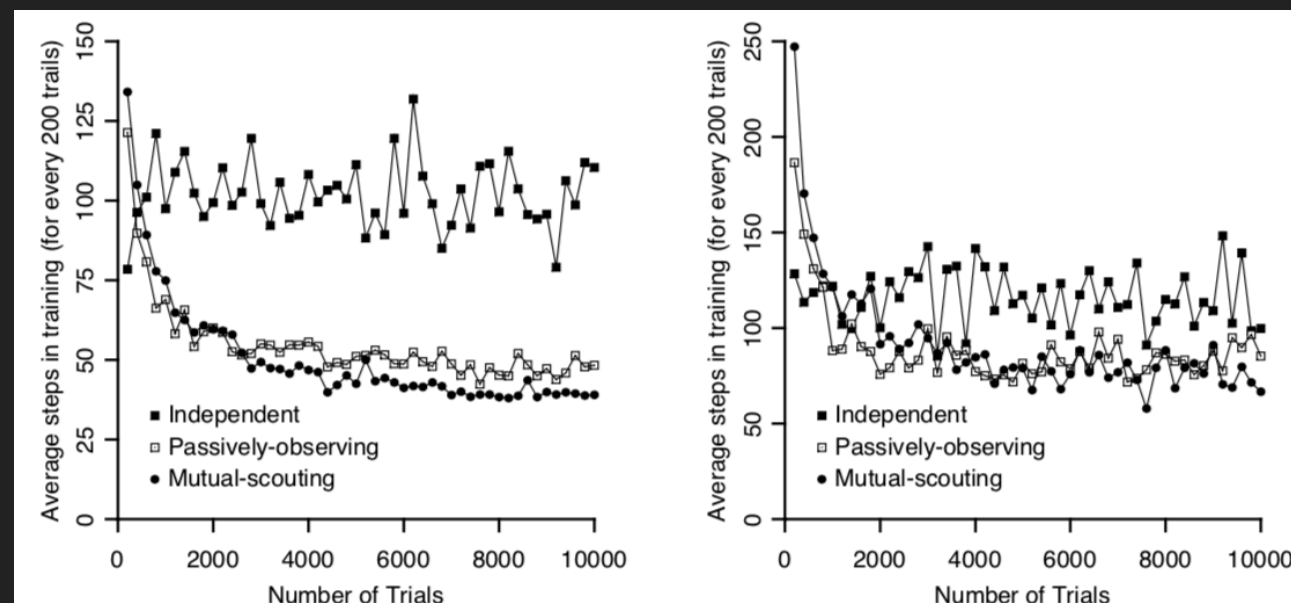
CASO 2 – COMPARTIR POLÍTICAS Y EPISODIOS

- ▶ Compartir conocimiento mas elaborado (políticas o experiencias o episodios) acelera el aprendizaje si se aprende de un experto.
- ▶ Tiene un costo alto de comunicación.



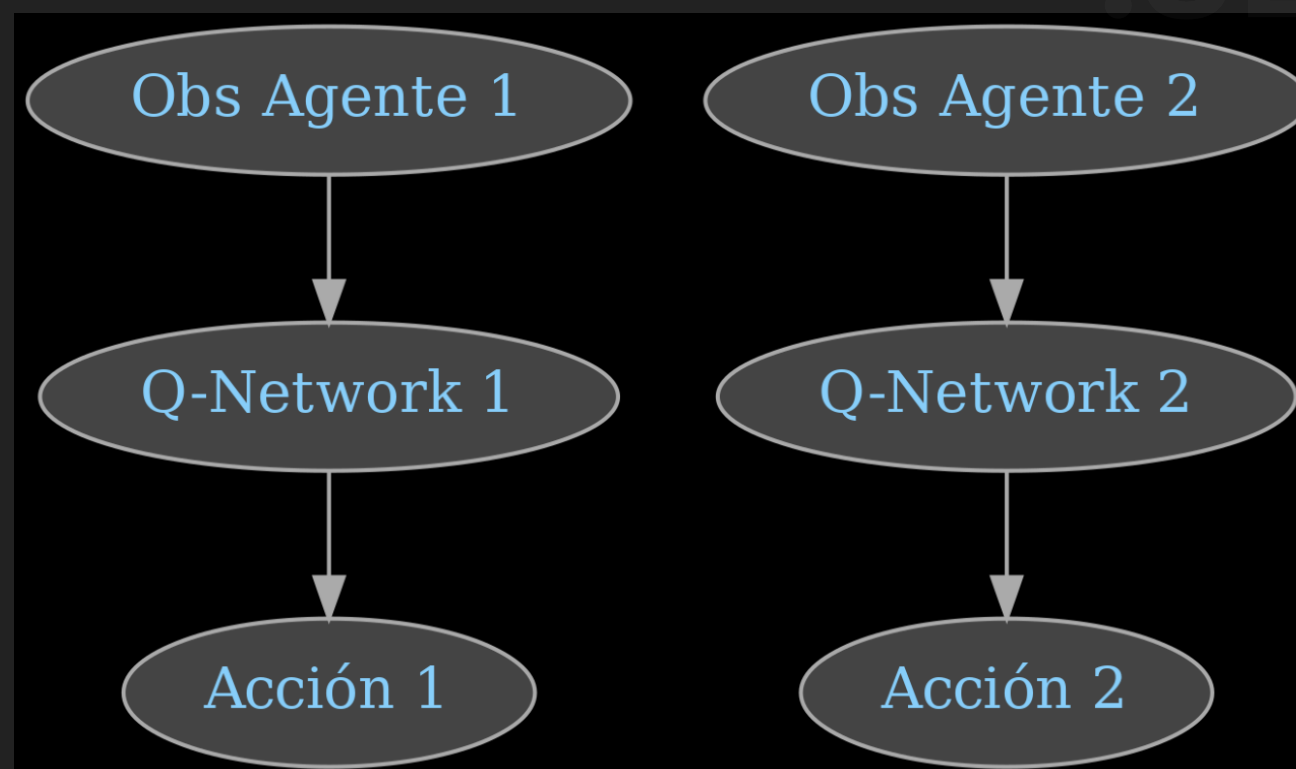
CASO 3 – TAREAS CONJUNTAS

- ▶ Tareas conjuntas (o tareas que requieren cooperación si o si)
- ▶ Consiste en 2 cazadores contra 2 presas y 2 cazadores o con 1 presa.
- ▶ IQL falla (persiguen presas distintas o no se coordinan para acorralar a la misma).
- ▶ Se soluciona permitiendo percibir donde esta el otro cazador, además de la presa. (el estado incluye al peer), con eso aprendieron a cooperar.



VENTAJA Y DESVENTAJA

- ▶ Ventaja: Simple y fácil de implementar.
- ▶ Desventaja: Puede fallar si los agentes necesitan cooperar, porque no entienden el impacto de los demás en el entorno.



IQL

- ▶ Robótica colaborativa
- ▶ Sistemas distribuidos.
- ▶ Flota de vehículos autónomos.

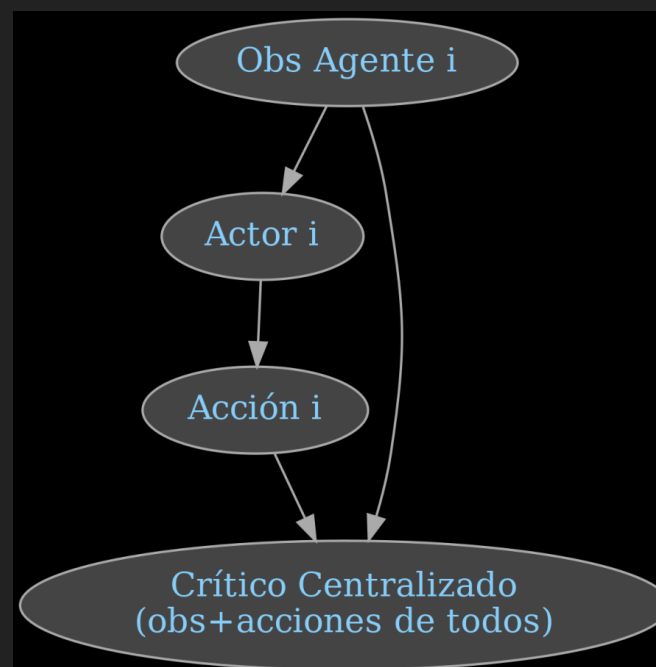


MADDPG



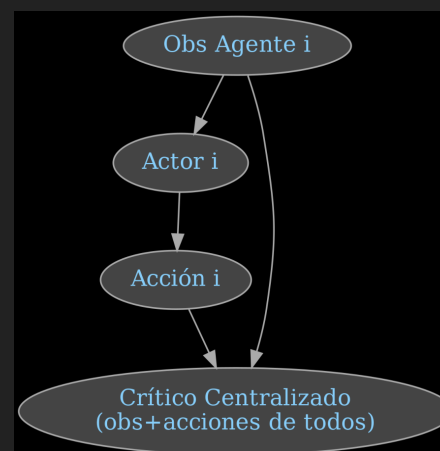
MADDPG

- ▶ Cada agente tiene su propia estrategia (policy), pero cuando se entrena, usa la información de TODOS los jugadores (incluso enemigos) para mejorar.
- ▶ Sirve cuando se busca que cada agente actúe con base en observaciones parciales, pero que aprendan con información global (durante el entrenamiento).
- ▶ ¿cómo se logra que varios agentes aprendan al mismo tiempo cuando el éxito de un agente depende de los otros o cuando compiten entre ellos?



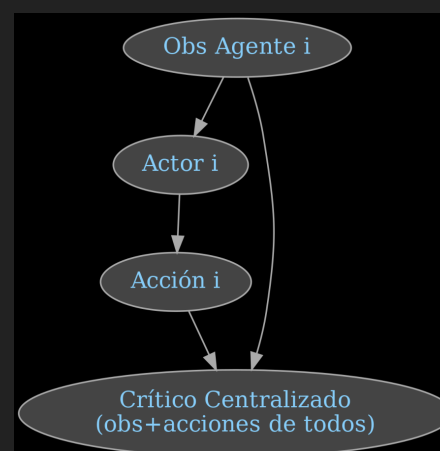
MADDPG

- ▶ MADDPG propone un algoritmo que busca servir para todo, **cooperación, competición, mixto**, sin conocer el entorno y sin asumir que los agentes ya saben comunicarse.
- ▶ Los algoritmos para un agente (SARL) fallan cuando hay varios agentes a la vez.



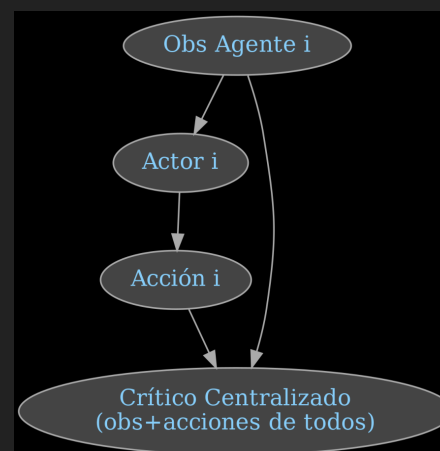
PROBLEMÁTICA: NO ESTACIONARIEDAD

- ▶ ¿porque es tan difícil entrenar a varios agentes juntos? **No estacionariedad**
- ▶ Si un agente esta intentando aprender algo y a su vez si los demás también lo hacen y están cambiando sus estrategias todo el tiempo, desde el punto de vista del primer agente **las reglas cambian constantemente de forma impredecible.**
- ▶ El comportamiento de los otros no para de evolucionar y eso vuelve el entorno inestable, esa inestabilidad (**no estacionariedad**) es un problema para los métodos clásicos de RL.



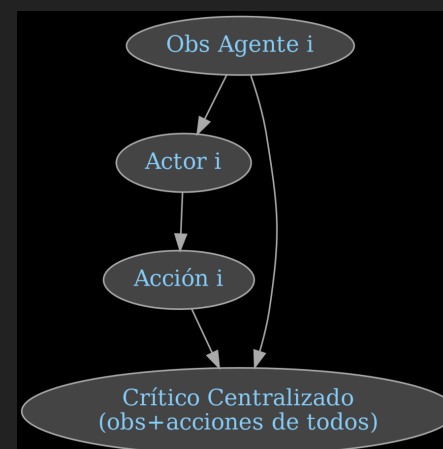
NO ESTACIONARIEDAD EN DQN

- ▶ Q-learning y DQN aprenden el valor de hacer x cosa en y situación, pero si la situación real incluye las **estrategias cambiantes de los demás**, la base de que el entorno es predecible (propiedad de Markov) se rompe. El aprendizaje se vuelve errático, cuesta mucho aprender algo útil.
- ▶ Esto **afecta** cosas como la **memoria de repetición** de DQN donde se guardan experiencias pasadas para aprender de ellas y para estabilizar dado que **si los otros agentes ya cambiaron su forma de actuar desde que se guardó una experiencia recordar el pasado puede ser inútil o puede confundir**. Se usa algo que ya no vale.



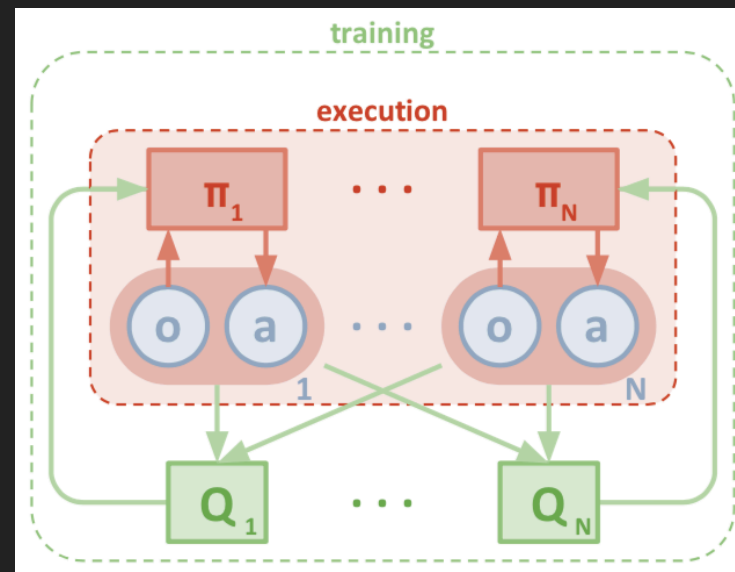
EL CRÉDITO O LA CULPA

- ▶ En los métodos PG que ajustan la estrategia directamente, si algo fue bueno o malo, el gradiente se vuelve **ruidoso** (mucha varianza) y es difícil saber que parte del **mérito** es de cuál agente. El **crédito** o la **culpa** se diluye mucho. Es difícil saber si ajustar la estrategia en una dirección es bueno o si solo se tuvo suerte o mala suerte con lo que hicieron los demás.
- ▶ **Cuanto más agentes peor**, más agentes => más ruido, más difícil es saber que funciona. El aprendizaje se vuelve lento e inestable.
- ▶ Un agente solo aprende bien en un mundo predecible pero muchos agentes aprendiendo a la vez se vuelven impredecibles entre sí.



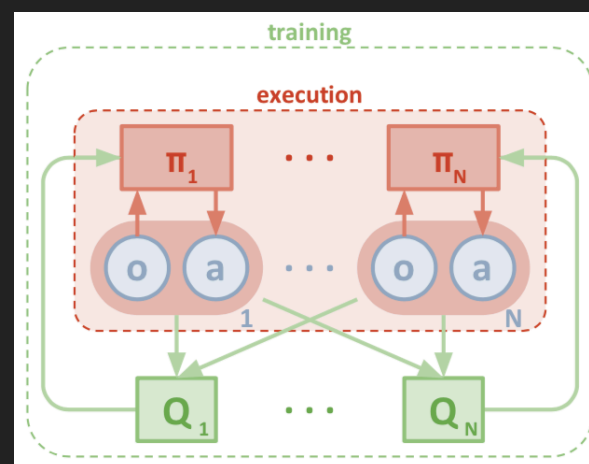
¿QUÉ PROPONE MADDPG?

- ▶ En MADDPG se propone usar entrenamiento centralizado con ejecución descentralizada CTDE.
- ▶ En futbol sucede algo similar (entrenador y jugadores).
- ▶ En MADDPG cada agente tiene 2 estructuras:
 - ▶ **Actor**: decide qué hacer (acción) y decide basado en lo que ve localmente.
 - ▶ **Crítico**: evalúa si la acción del actor fue buena o mala.



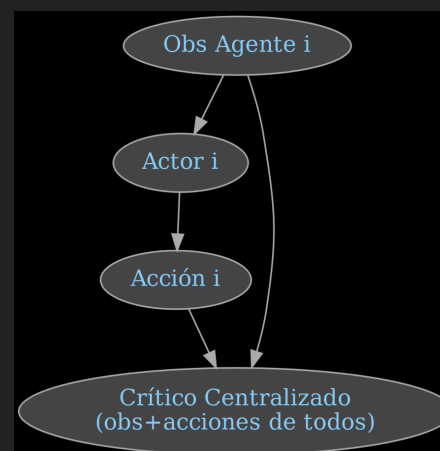
COMO FUNCIONA MADDPG

- ▶ En MADDPG durante el **entrenamiento** el critico de cada agente recibe **información extra**, no solo lo que observo el agente sino también información sobre las acciones y observaciones de todos los demás agentes.
- ▶ Esto es **temporal**, al tener visión global solo durante el entrenamiento el critico puede aprender a valorar las acciones de cada actor de forma más fiable.
- ▶ Para el critico (que ve todo el entorno) el entorno le parece **estable** porque conoce las acciones de todos que causan los cambios, **así supera el problema de la no estacionariedad** y puede aprender a juzgar bien qué es bueno y que es malo.
- ▶ Luego, el **actor** que juega de verdad en la **ejecución** nunca tiene acceso a esa información extra, decide solo con visión local, **se beneficia de haber sido entrenado por un critico que estaba mucho mejor informado**.
- ▶ Como cada agente aprende su propia función de crisisco aunque use información centralizada pueden tener **objetivos distintos**, funciona si cooperan, si compiten o si cada uno va a lo suyo, no asume que todos buscan lo mismo.



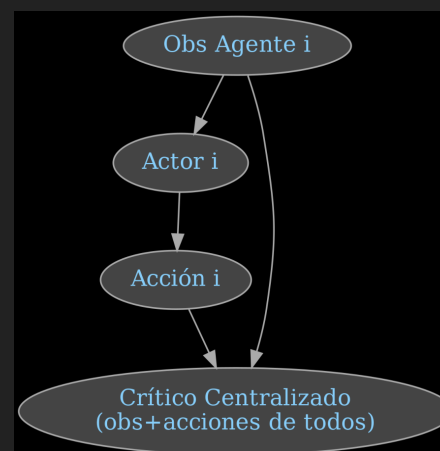
MADDPG

- ▶ Durante el entrenamiento un agente no puede saber qué hacen exactamente los otros, ve sus acciones pero no conoce su estrategia interna.
- ▶ MADDPG propone **inferir políticas ajenas**: si no se puede ver dentro de los otros agentes, cada agente puede intentar aprender un modelo de como creen que los demás toman decisiones, deducen la estrategia del otro y ajustan un modelo interno para predecir que harán, **"el agente B se movió así, de modo que su estrategia es esta"**, y usan esas predicciones (modelos aproximados) para informar a su critico centralizado.
- ▶ MADDPG funciona bien en cooperación.



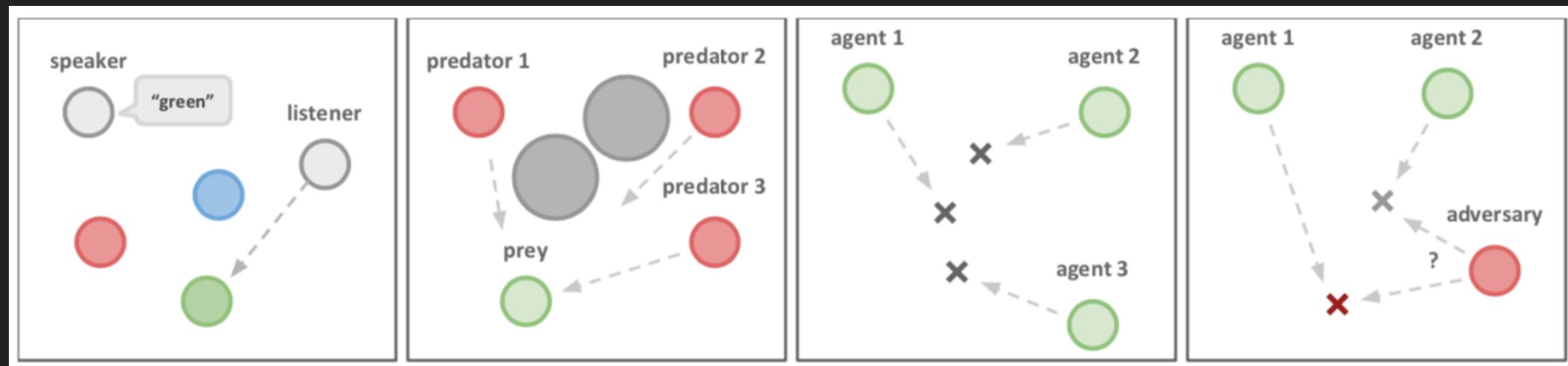
MADDPG EN COMPETENCIA

- ▶ Si un agente se vuelve muy bueno contra un rival específico pero si ese rival cambia un poquito su juego el agente se pierde (**sobreajuste**), se especializa demasiado en un oponente en concreto.
- ▶ Para evitar eso MADDPG propone los “**ensambles de políticas**”, en vez de aprender una única estrategia, **cada agente aprende una colección varias sub-políticas distintas** (k estrategias diferentes).
- ▶ En cada episodio de entrenamiento se **eligen** aleatoriamente **una de esas k sub-políticas** para que el agente la use en esa ronda y se entrena toda la colección junta. Es obligar al agente a ser **bueno no contra solo un estilo** sino contra varios estilos diferentes representados por las distintas sub-políticas de los otros, se lo fuerza a ser versátil al funcionar bien contra una variedad de estrategias, sean estas de rivales o de compañeros. Evita que se encasille.



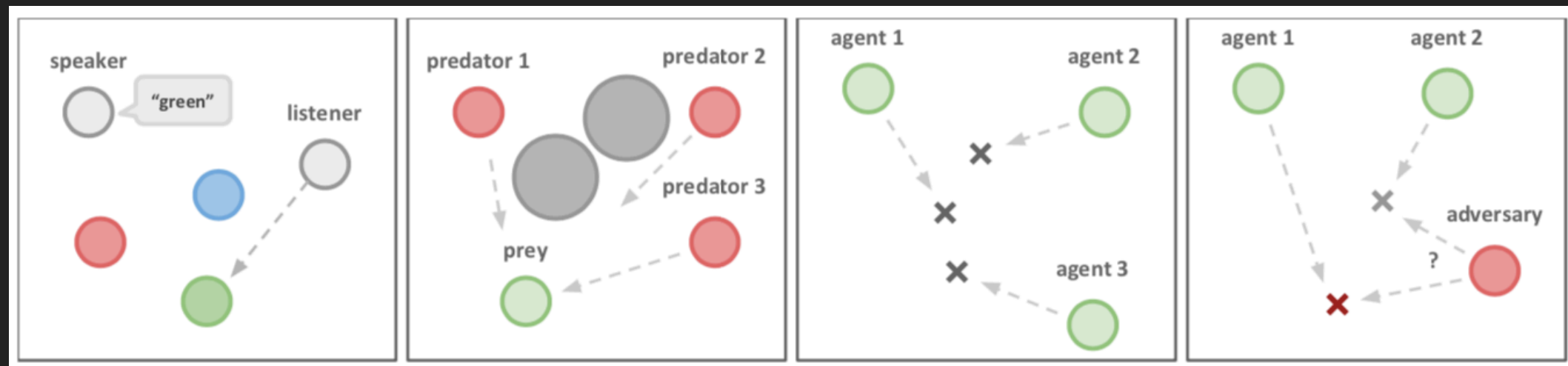
MADDPG EN COMUNICACIÓN COOPERATIVA

- ▶ Hay 2 agentes, un **hablante** y un **oyente**, varios **landmarks** de colores.
- ▶ El hablante sabe a qué color tiene que ir el oyente, pero el oyente no, el oyente no sabe cuál color es el bueno. El hablante tiene que de alguna forma decirle al oyente adónde ir.
- ▶ El hablante emite una señal (código) que representa el color correcto y el oyente aprende a interpretar esa señal y a moverse al landmark correcto (84% de las veces es exitoso).



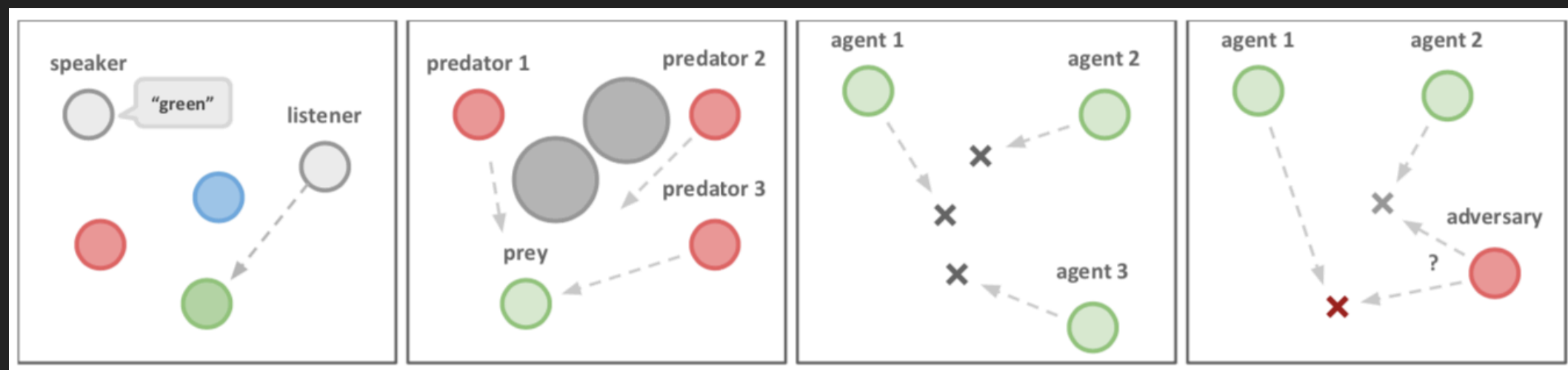
MADDPG EN ENGAÑO FISICO

- ▶ Varios agente cooperativos quieren llegar a un landmark objetivo y hay un adversario también quiere llegar pero **no sabe cuál es el objetivo correcto**. Los cooperativos si lo saben. Para ganar los coop. tienen que llegar y confundir al adversario.
- ▶ Los agentes coop. se coordinan para cubrir todos los landmarks a la vez no solo el verdadero así el adversario no tiene idea a cual ir (94% de las veces exitoso).



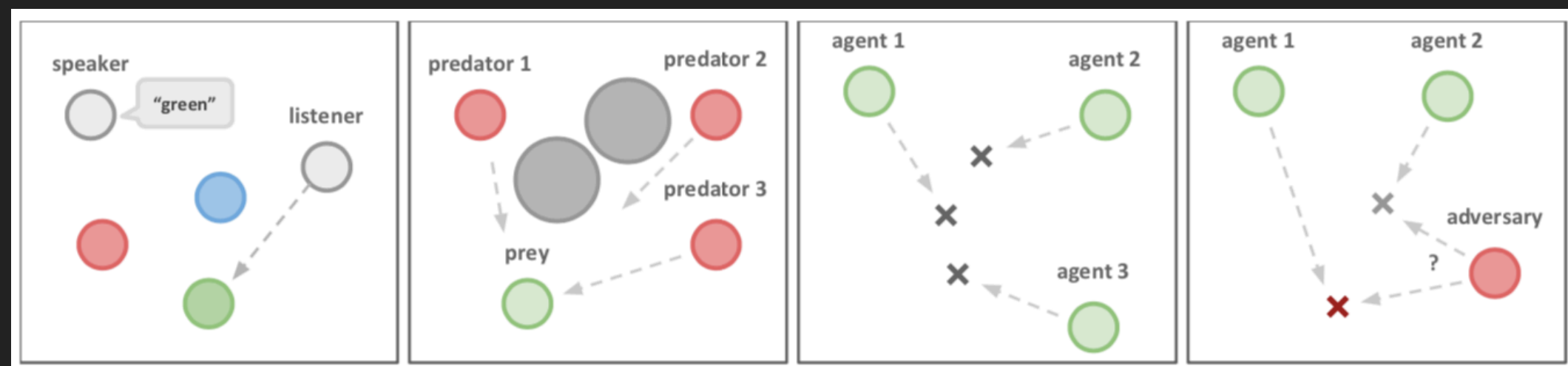
MADDPG EN DEPREDADOR PRESA

- ▶ Varios depredadores lentos cooperan para atrapar a una presa rápida. Los depreda. capturan presas muchas mas veces, (16 colisiones por episodio).



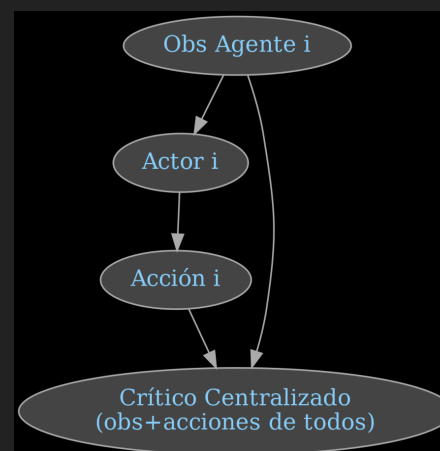
MADDPG EN COMUNICACIÓN ENCUBIERTA

- ▶ A quiere enviar un mensaje a B, C intenta interceptarlo. A y B comparten una clave secreta. A tiene que codificar el mensaje para que B lo entienda pero C no.
- ▶ MADDPG ayuda a B a entender mejor el mensaje también permite a A aprender a codificar mejor el mensaje de forma que para C fuera casi imposible descifrarlo.



APLICACIONES

- ▶ Flota de drones
- ▶ Modelado de sistemas económicos o sociales complejos
- ▶ Control de robots
- ▶ Análisis de dilemas sociales

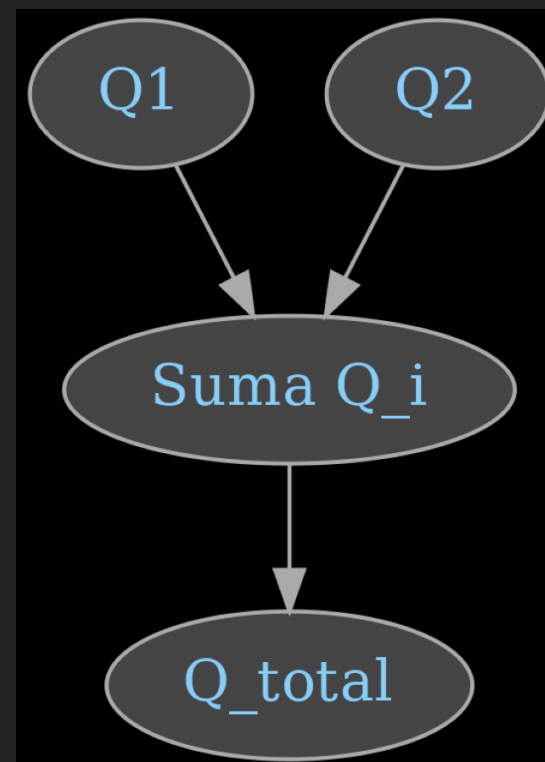


VDN



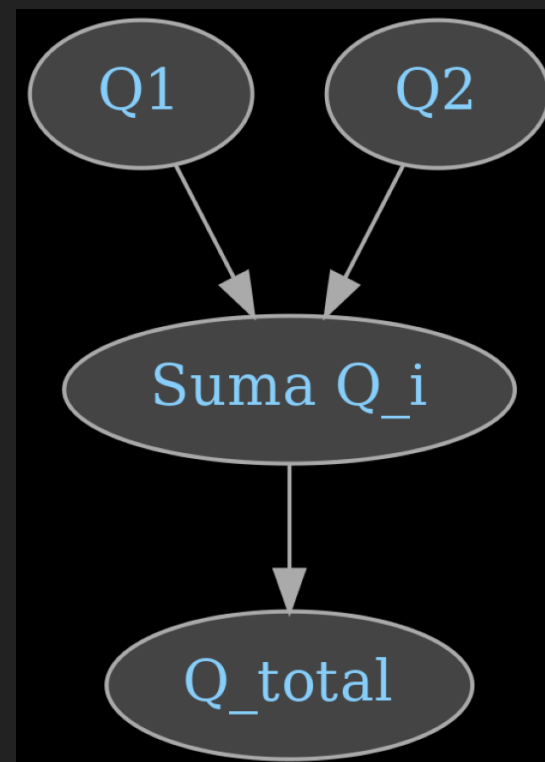
VDN

- ▶ Redes de Descomposición de Valor (Value-Decomposition Networks)
- ▶ Asume que el valor total (recompensa) del equipo se puede descomponer como la suma de lo que cada agente contribuye.
- ▶ Cada agente aprende su propia "parte" del valor total.
- ▶ Ventaja: Fomenta cooperación simple, porque si todos mejoran, el equipo mejora.
- ▶ Desventaja: No sirve si la cooperación no es simplemente sumar (ej: tareas que requieren sincronización compleja).



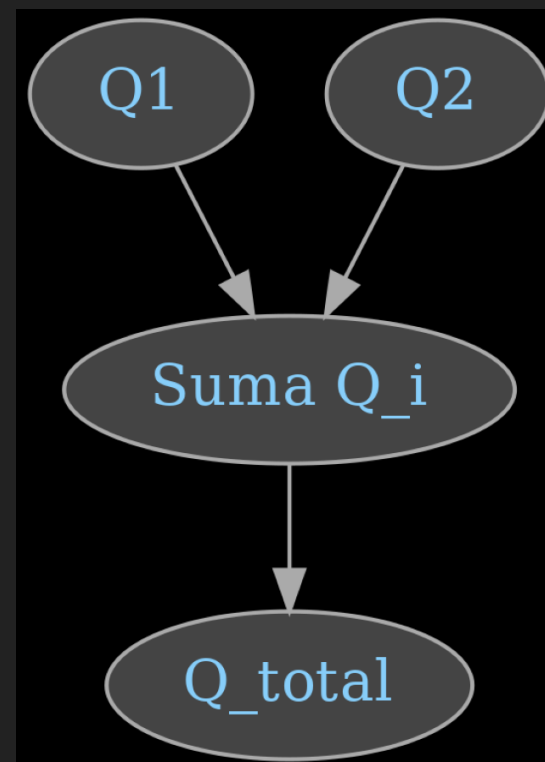
VDN

- ▶ Cada agente ve solo una parte del entorno (observaciones locales) y eligen que hacer (acciones)
- ▶ El objetivo es conseguir la mayor recompensa acumulada para el equipo.



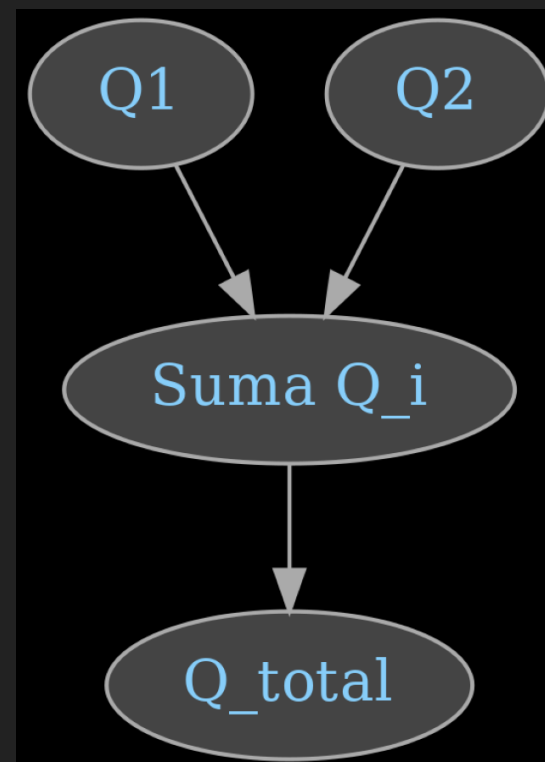
VDN

- ▶ Idea 1. Control centralizado.
- ▶ No funciona bien.
- ▶ 1. El numero de acciones conjuntas posibles crece exponencialmente con la cantidad de agentes. (se vuelve inmanejable)
- ▶ 2. El problema del agente perezoso. Si un agente aprende bien puede hacer que otro no intente aprender nada. El algoritmo aprende que el segundo agente se quede quieto para conseguir mejores resultados.



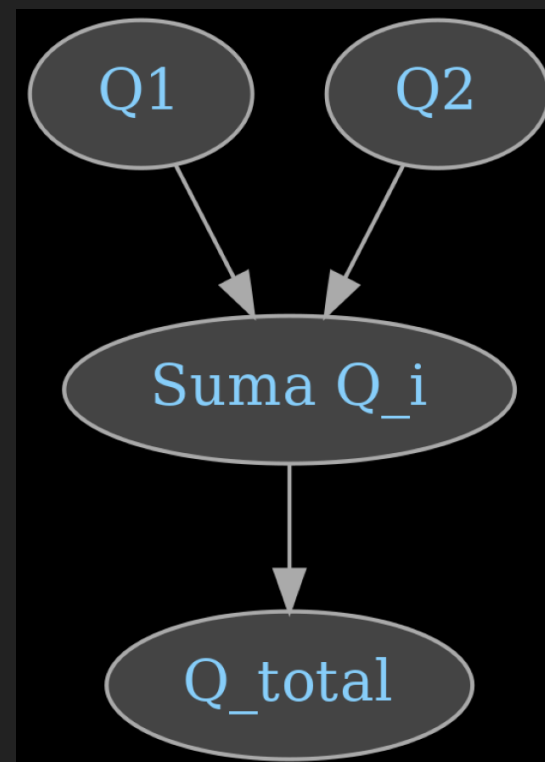
VDN

- ▶ Idea 2. Cada uno aprende por su cuenta.
- ▶ Para un agente individual el entorno cambia todo el tiempo aleatoriamente porque los demás agentes también cambian.
- ▶ Tiene el problema de a quien darle el merito. Si el equipo consigue una recompensa ¿como sabe un agente que fue por algo que hizo el o por algo que hizo otro?



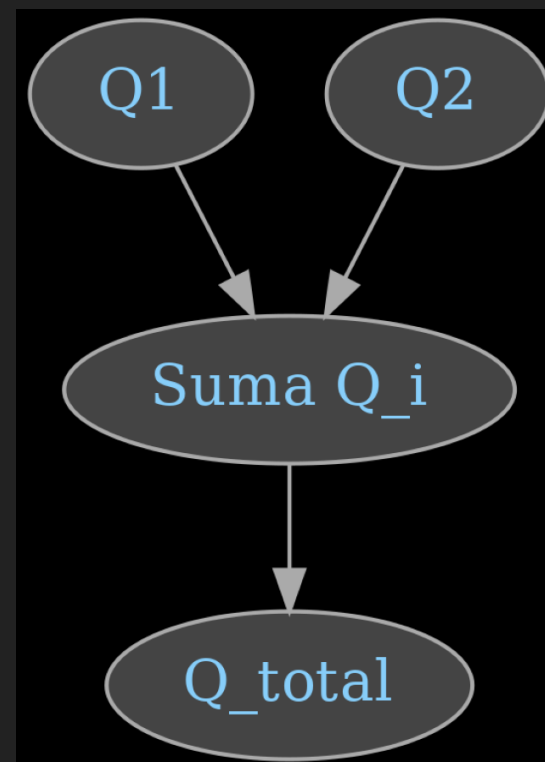
VDN

- ▶ Aparte de superar esto VDN propone conectar el valor de la acción de todo el equipo (Q_{total}) con los valores de las acciones individuales de cada agente.



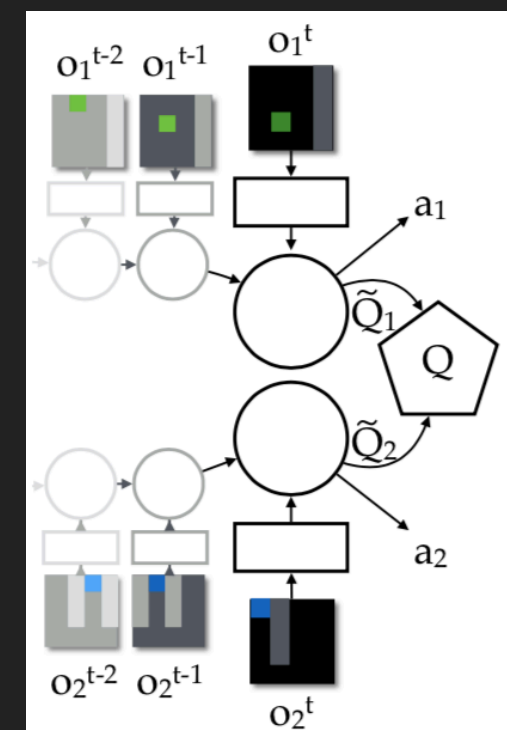
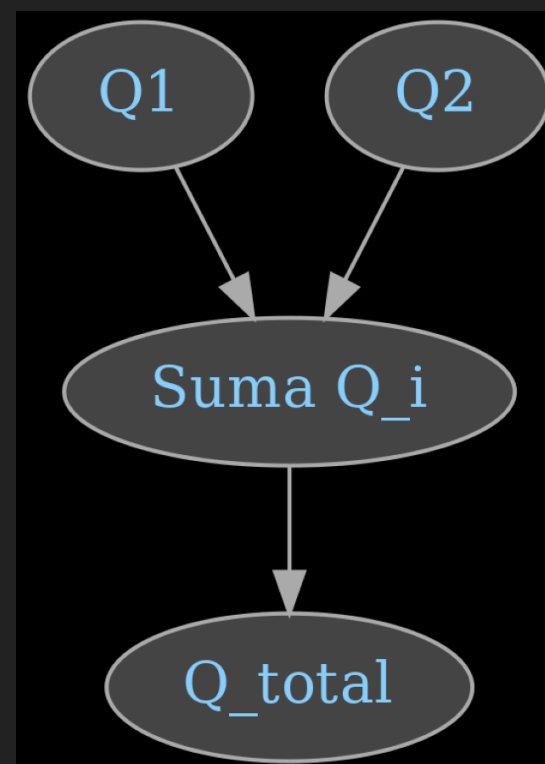
VDN

- ▶ Si la única señal de aprendizaje es la recompensa global ¿cómo aprenden las Q_i individuales?
- ▶ Las Q_i se aprenden solas mediante retropropagación. El algoritmo calcula cómo ajustar cada Q_i para que cuando se sumen todas el resultado se parezca lo más posible al valor real que obtuvieron basado en la recompensa.
- ▶ Durante la ejecución puede ser descentralizada. Cada agente mira su entorno local, calcula su propia Q para las acciones que hace y elige la más alta.
- ▶ Evita las recompensas espurias del aprendizaje independiente porque cada Q_i usa información relevante para ese agente.



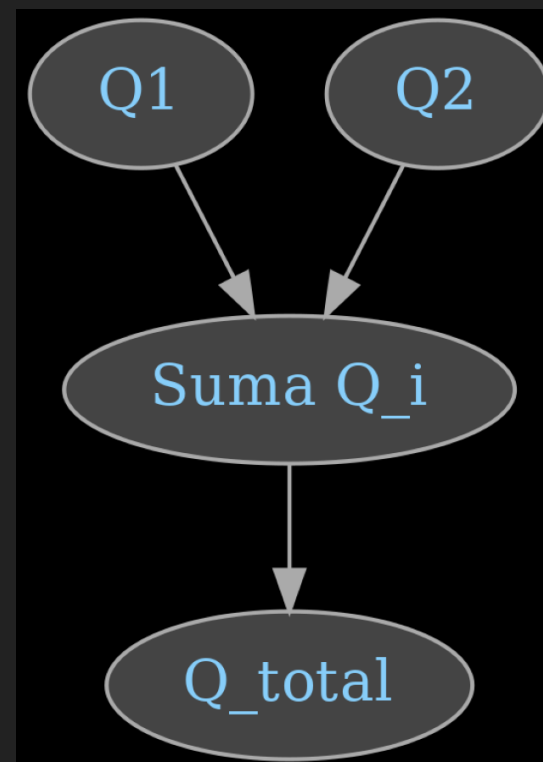
VDN

- ▶ Que ocurriría si lo que hace un agente solo tiene valor si otro hace algo específico al mismo tiempo? Una suma simple no capta eso bien.
- ▶ Ese es el punto débil. La descomposición aditiva (suma) limita la complejidad de lo que la red puede representar. No modela esas dependencias cruzadas directas y no lineales.
- ▶ Pero para muchas tareas esta aproximación es buena. Ofrece ventajas en aprendizaje y estabilidad que compensa esa limitación.
- ▶ Value-decomposition individual architecture showing how local observations enter the networks of two agents over time (three steps shown), pass through the low-level linear layer to the recurrent layer, and then a dueling layer produces individual "values" that are summed to a joint Q-function for training, while actions are produced independently from the individual outputs.



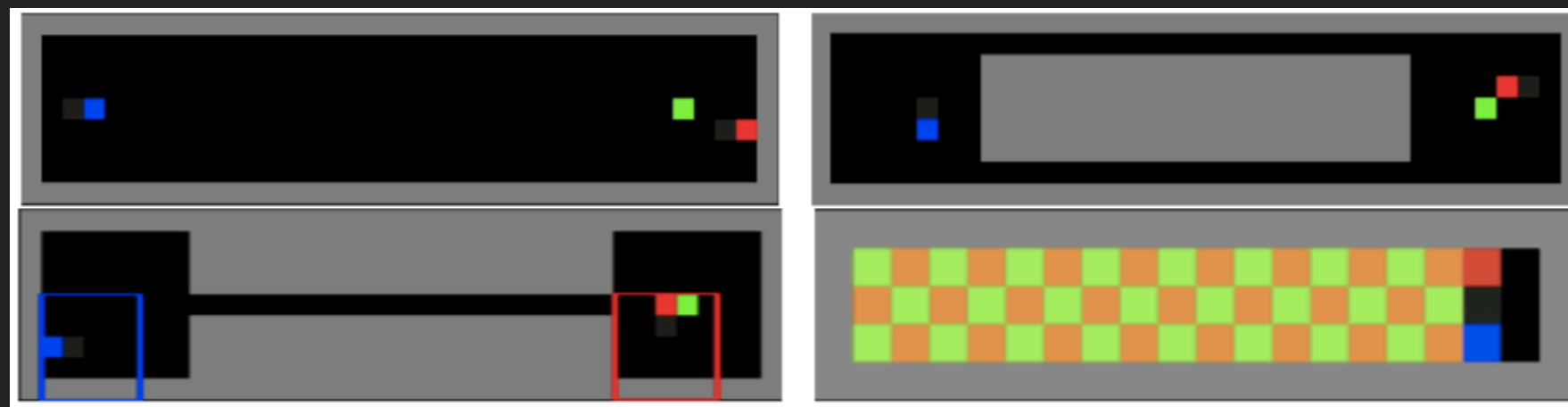
VDN

- ▶ VDN se evaluó de varias formas. La básica y combinada con otras técnicas.
- ▶ 1. Comparar pesos. Reduce mucho lo que debe aprender y funciona si los agentes no necesitan roles muy diferentes. Usa Invarianza vía agente (da igual que agente se use si son iguales)
- ▶ 2. Información de rol. Aunque compartan la red a veces si se necesita que se especialicen. Se le da a cada agente una entrada extra además de lo que ve (1-hot encoding)
- ▶ 3. Canales de información. Conexiones dirigidas entre las redes de los agentes que también se aprenden para compartir información interna ya procesada antes de decidir la acción final.



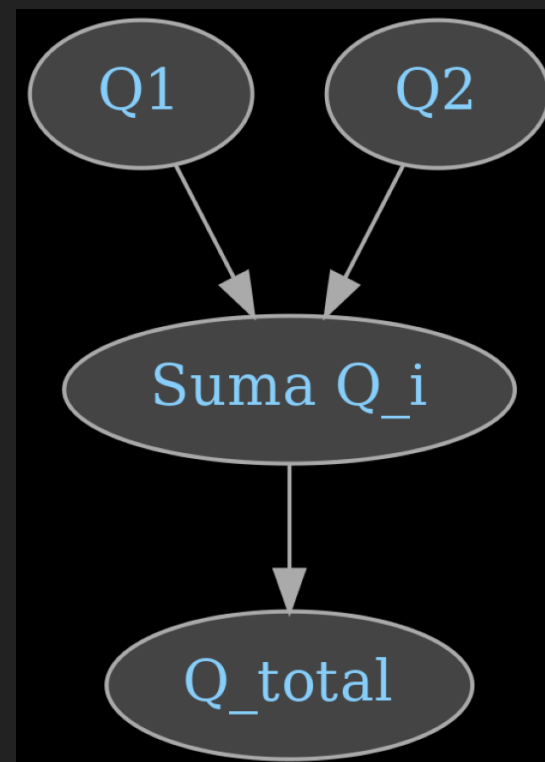
VDN

- ▶ Se aplicó a entornos 2D:
- ▶ Switch (corredor aboerto, 1 corredor, 2 corredores). Coordinación espacial. 2 agentes (uno en cada extremo) deben llegar a la meta opuesta. Si se encuentran deben ponerse de acuerdo de quien pasa primero.
- ▶ Fetch (corredor aboerto, 1 corredor, 2 corredores). Sincronización en el tiempo. 2 agentes recogen objetos de un lado y los llevan a una base en el otro. Ganan puntos al recoger y al entregar. Lo optimo es que sincronicen mientras uno va el otro vuelve.
- ▶ Checkers. Prueba la especialización. Manzanas dan +10 y limones quitan -10. 2 agentes con distinta sensibilidad. Hay limones bloqueando las manzanas. El sensible va por las manzanas y el otro ayuda quitando limones pero sin comerse las manzanas



APLICACIONES

- ▶ Sistemas complejos que colaboren adecuadamente
- ▶ Robots
- ▶ Sensores
- ▶ Equipos humanos

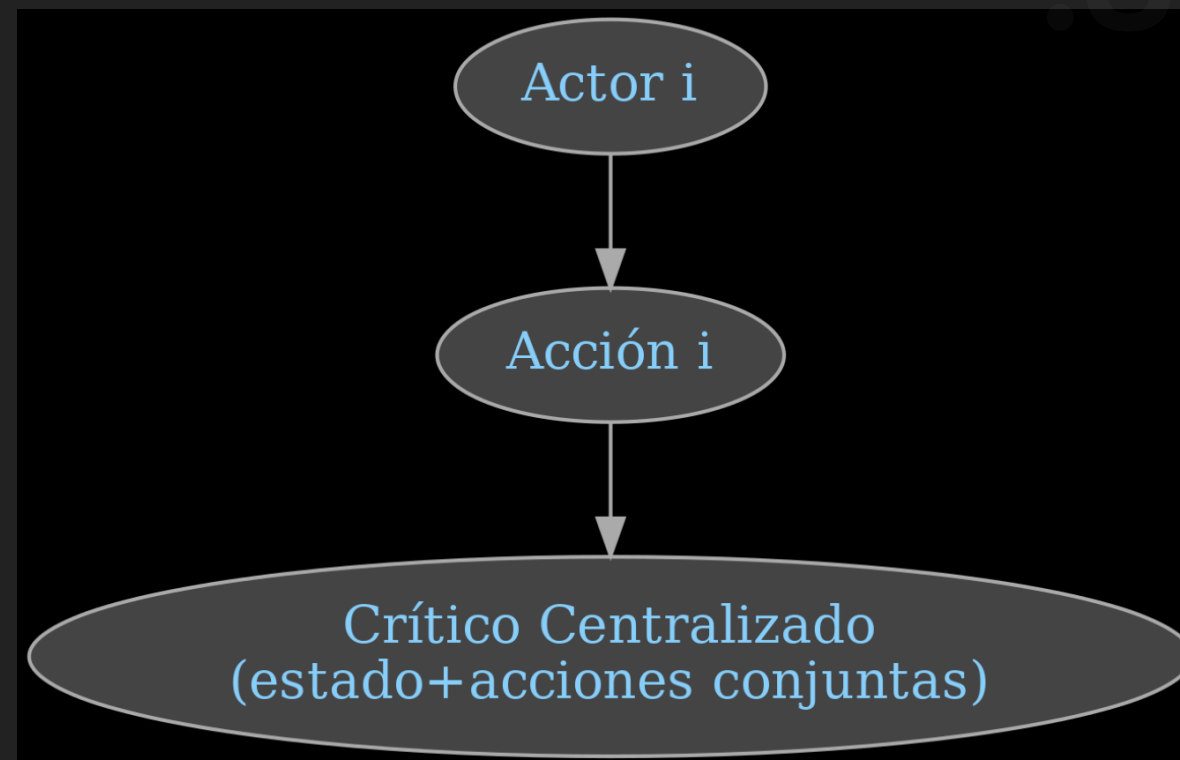


COMA



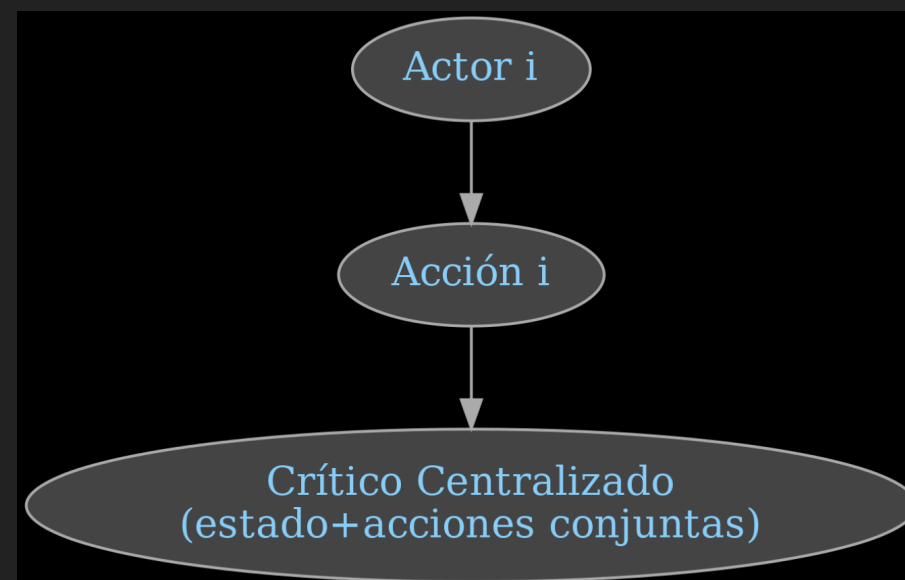
COMA (COUNTERFACTUAL MULTI-AGENT)

- ▶ Cuando cada agente tiene una visión parcial del entorno
¿Cómo se asegura que el grupo tenga éxito si lo que hace un agente puede afectar a todos los demás?



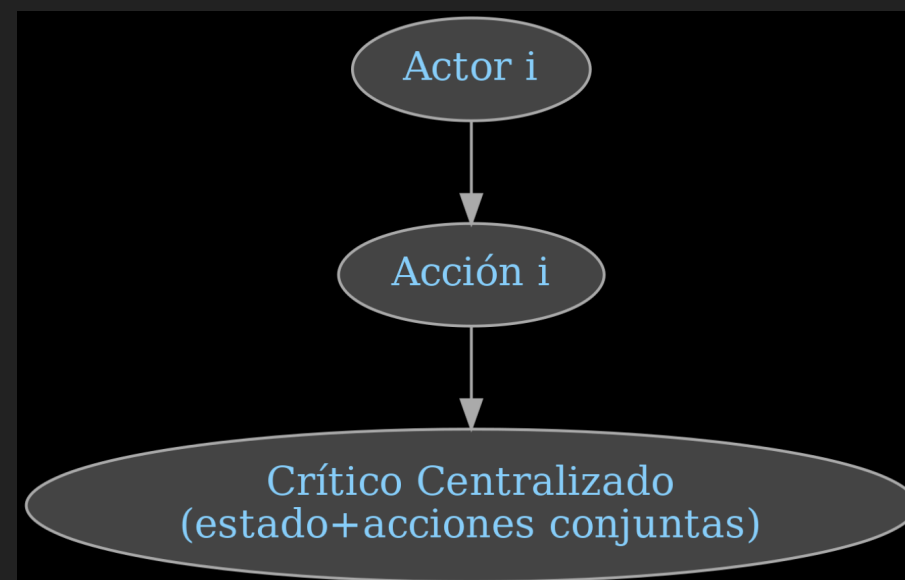
COMA (COUNTERFACTUAL MULTI-AGENT)

- ▶ Usa un artificio llamado "contrafactual":
- ▶ Para cada agente i calcula cuánto mejor o peor habría sido el resultado si i hubiera actuado diferente, asumiendo que los demás hacen lo mismo
- ▶ Así, cada agente entiende su impacto personal (visión parcial) en el resultado global.



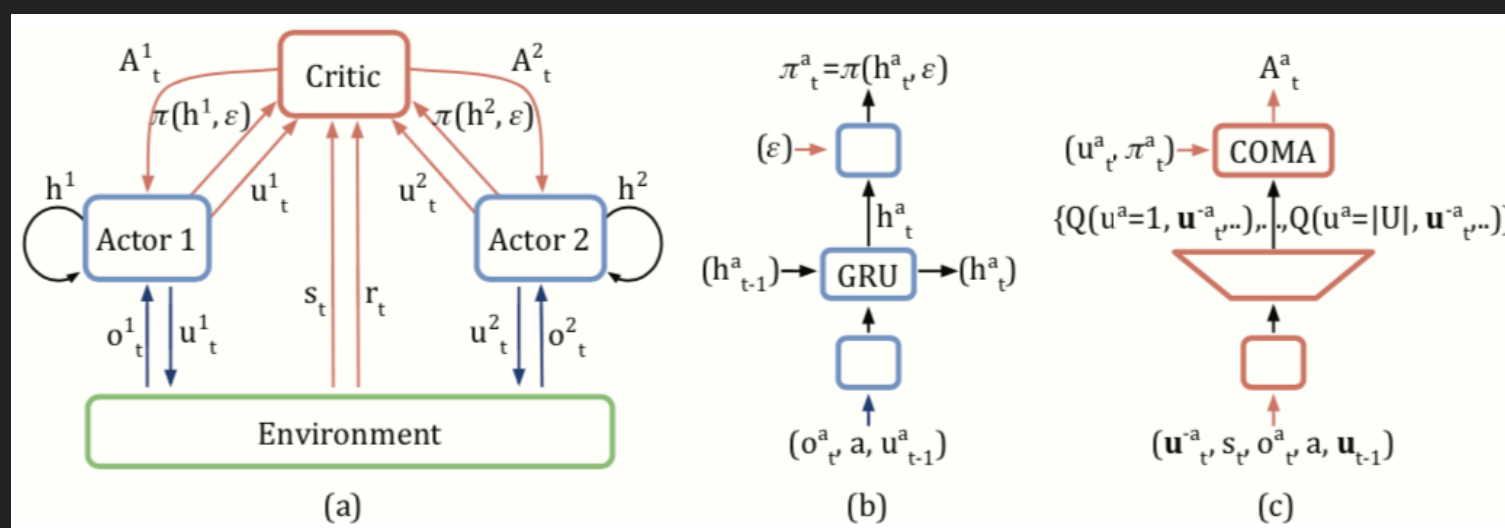
COMA (COUNTERFACTUAL MULTI-AGENT)

- ▶ Crítico centralizado en el entrenamiento
- ▶ Baseline contrafactual (para el crédito individual)
- ▶ Arquitectura de red eficiente



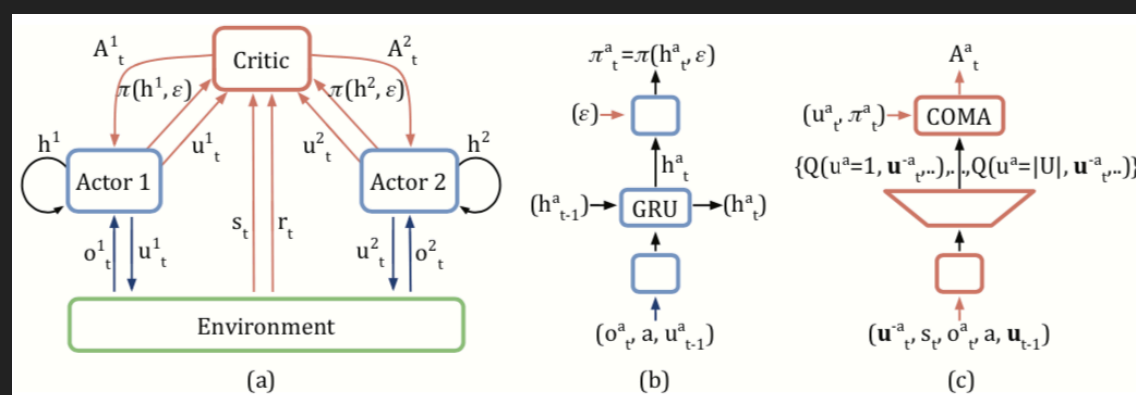
CRÍTICO CENTRALIZADO EN EL ENTRENAMIENTO

- ▶ COMA tiene varios actores y un critico.
- ▶ Se usa entrenamiento centralizado y ejecución descentralizada **CTDE**.
- ▶ El actor usa la critica para ajustar sus próximos movimientos.
- ▶ Se le otorga al critico mas información durante el entrenamiento.
- ▶ Los actores operan descentralizados con visión limitada.



BASELINE CONTRAFACTUAL (PARA EL CRÉDITO INDIVIDUAL)

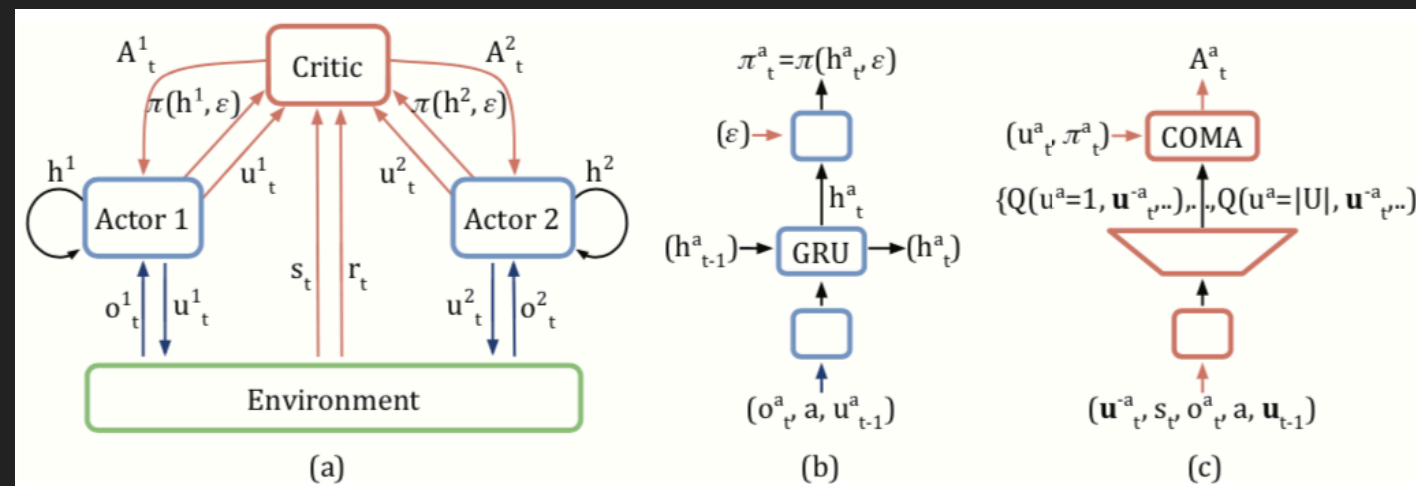
- ▶ ¿Cómo ayuda el critico centralizado a resolver la asignación de crédito?
- ▶ Lo hace mediante el concepto llamado "contrafactual".
- ▶ Compara el resultado que se obtuvo con el resultado que se habría obtenido si solo ese agente hubiera hecho otra cosa, pero los demás hubieran hecho lo mismo <- difference rewards.
- ▶ Lo hace usando información que ya tiene el crítico centralizado. El critico estima que tan bueno es tomar ciertas acciones conjuntas en un estado dado y con esa función Q aprendida puede calcular la linea de base contrafactual (baseline) de forma eficiente.



$$D^a = r(s, \mathbf{u}) - r(s, (\mathbf{u}^{-a}, c^a))$$

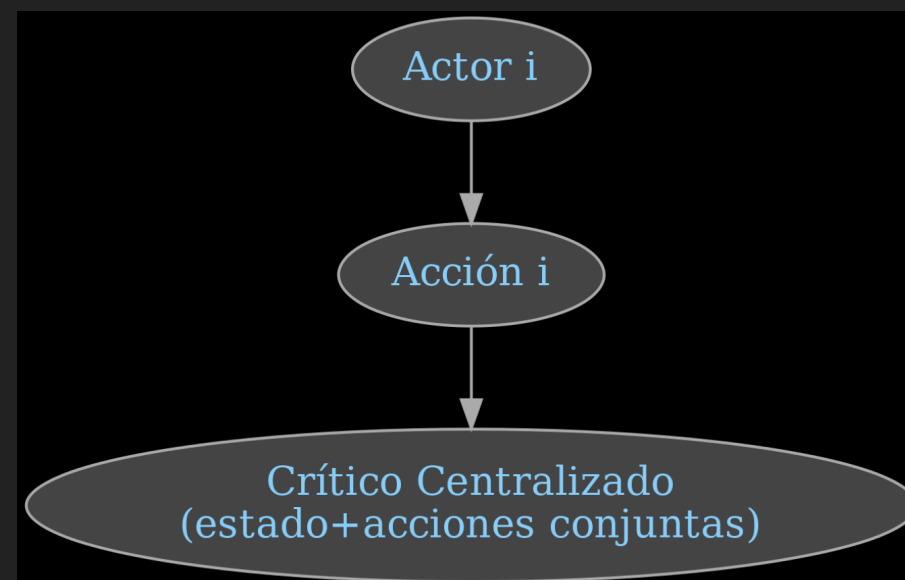
ARQUITECTURA DE RED EFICIENTE

- ▶ El crítico recibe como **entrada** el estado global y las acciones que tomaron todos los agentes.
- ▶ La **salida** del crítico es un conjunto de valores, uno por cada acción posible de cada agente específico que esta evaluando en ese momento (para cada agente calcula solo sus posibilidades de acciones).



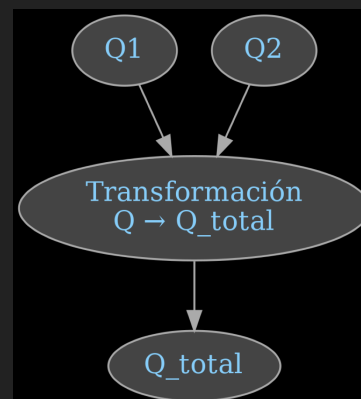
VENTAJA Y DESVENTAJA

- ▶ Ventaja: Es muy bueno para dar crédito justo a cada agente.
- ▶ Desventaja: Es costoso computacionalmente, necesita simular muchas alternativas.



QTRAN

- ▶ Cada agente aprende su propia “**tabla de valores**” (Q individual).
- ▶ **QTRAN** transforma esas tablas individuales en un valor global usando una red neuronal libre (sin restricciones).
- ▶ Para evitar resultados caóticos, usa una función de pérdida especial que asegura que **la mejor acción del equipo = la combinación de las mejores acciones individuales**.
- ▶ Si cada uno de los agentes elige lo mejor para sí mismo (en contexto de equipo), el equipo gana.



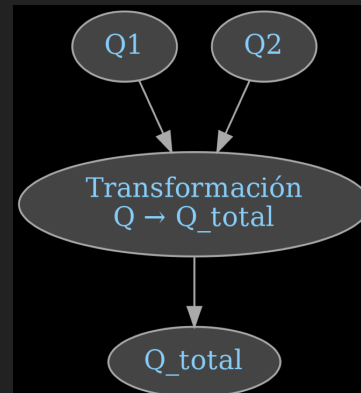
VENTAJAS Y DESVENTAJAS DE QTRAN

► Ventajas

- ✓ Puede aprender cualquier tipo de interacción entre agentes, por rara que sea.
- ✓ No tiene las limitaciones de VDN (solo suma) ni de QMIX (monotonicidad).
- ✓ Ideal para entornos donde la cooperación es no aditiva, no lineal, o contraintuitiva.

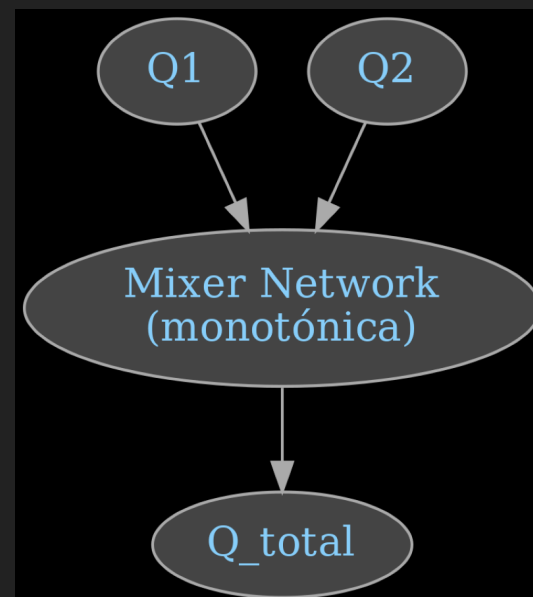
► Desventajas

- ✓ Es más difícil de entrenar (la función de pérdida es compleja y a veces inestable).
- ✓ Requiere más datos y más cómputo.
- ✓ En la práctica QMIX suele funcionar mejor que QTRAN en entornos comunes (como SMAC).



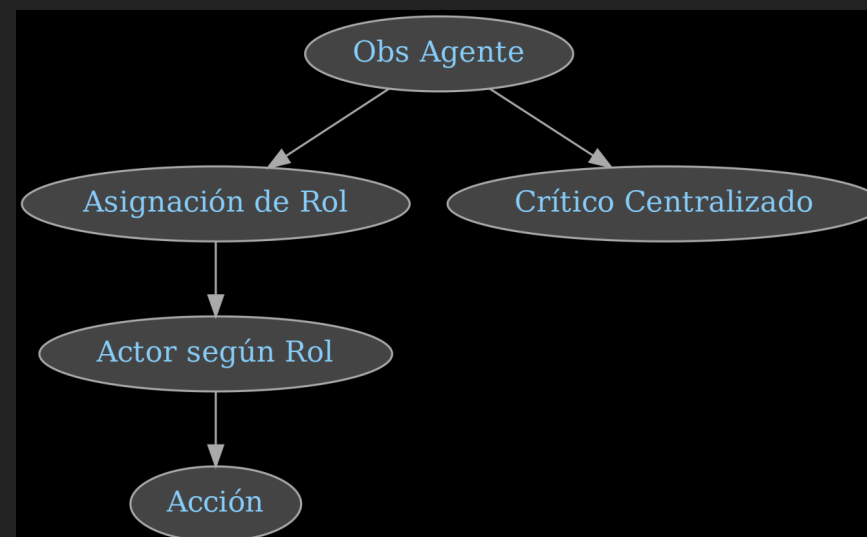
QMIX

- ▶ Es una mejora de **VDN**. En vez de sumar, combina los valores individuales con una red neuronal que aprende cómo mezclarlos de forma no lineal.
- ▶ Respeto la regla de que el valor global debe aumentar si los valores individuales aumentan (**monotonidad**).
- ▶ Ventaja: Es más flexible que VDN y maneja mejor tareas complejas.
- ▶ Desventaja: Más complicado de entrenar.



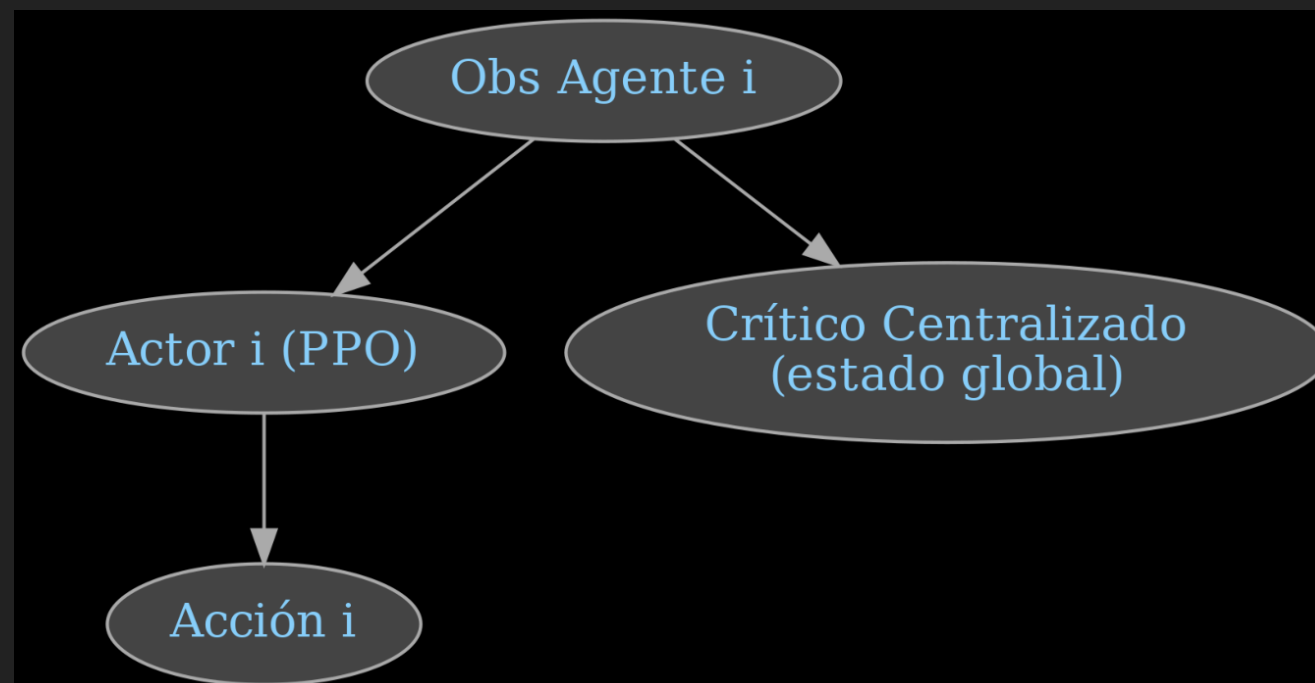
ROMA (MULTI-AGENT REINFORCEMENT LEARNING WITH EMERGENT ROLES)

- ▶ Asigna **roles** dinámicos a los agentes. Un rol define qué tipo de comportamiento debe tener.
- ▶ Los **roles pueden cambiar** con el tiempo según la situación.
- ▶ Ventaja: Ideal para entornos donde los agentes deben especializarse o cambiar de función.
- ▶ Desventaja: Hay que diseñar bien cómo se asignan y cambian los roles.



MAPPO (MULTI-AGENT PPO)

- ▶ Un entrenador (PPO) enseña a un equipo de agentes clones con la misma técnica, pero cada uno actúa en su posición.
- ▶ MAPPO tiene simplicidad y escalabilidad.
- ▶ Funciona bien en entornos complejos.



REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- ▶ Tan, M. (1993). Multi-agent reinforcement learning: Independent vs. cooperative agents. In Proceedings of the tenth international conference on machine learning (pp. 330-337).
- ▶ Sunehag, P., Lever, G., Gruslys, A., Czarnecki, W. M., Zambaldi, V., Jaderberg, M., ... & Graepel, T. (2017). Value-decomposition networks for cooperative multi-agent learning. arXiv preprint arXiv:1706.05296.
- ▶ Lowe, R., Wu, Y. I., Tamar, A., Harb, J., Pieter Abbeel, O., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. Advances in neural information processing systems, 30.
- ▶ Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., & Whiteson, S. (2018, April). Counterfactual multi-agent policy gradients. In Proceedings of the AAAI conference on artificial intelligence (Vol. 32, No. 1).
- ▶ Rashid, T., Samvelyan, M., De Witt, C. S., Farquhar, G., Foerster, J., & Whiteson, S. (2020). Monotonic value function factorisation for deep multi-agent reinforcement learning. Journal of Machine Learning Research, 21(178), 1-51.
- ▶ Son, K., Kim, D., Kang, W. J., Hostallero, D. E., & Yi, Y. (2019, May). Qtran: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In International conference on machine learning (pp. 5887-5896). PMLR.
- ▶ Wang, T., Dong, H., Lesser, V., & Zhang, C. (2020). Roma: Multi-agent reinforcement learning with emergent roles. arXiv preprint arXiv:2003.08039.
- ▶ Yu, C., Velu, A., Vinitisky, E., Gao, J., Wang, Y., Bayen, A., & Wu, Y. (2022). The surprising effectiveness of ppo in cooperative multi-agent games. Advances in neural information processing systems, 35, 24611-24624.